Assignment: Django Wallet-Based Per-Minute Session Billing

Stack: Django + Django REST Framework + PostgreSQL (SQLite is acceptable for local development)

Objective

Build a small backend system for a paid 1:1 consultation session.

The system should allow a user to:

1. Maintain a prepaid wallet balance.
2. Create a consultation session with a provider.
3. Start and end the session through a defined state machine.
4. Get charged from the wallet on a per-minute basis.
5. Automatically end the session when the wallet cannot cover the next minute.

The focus of this assignment is **state management, wallet handling, billing logic, and testing**.

---

1. Wallet

Create a wallet for each user with:

- `user`
- `balance`

The wallet should support:

- Viewing the current balance
- Adding money through a mock recharge
- Deducting money for session billing

Rules

- Wallet balance must never become negative.
- If a user does not have sufficient balance for a session, the session should not start.
- The minimum balance required to start a session is **5 × the per-minute session cost**.

---

2. Mock Wallet Recharge

Implement a simple mock recharge flow.

A user should be able to initiate a recharge and then complete it through a mock callback/webhook.

The recharge transaction should track:

- User
- Amount
- Reference
- Status

Possible statuses:

- `pending`
- `success`
- `failed`

Important

The same successful callback may be received multiple times.

The wallet must be credited **only once** for the same transaction.

No real payment gateway integration is required.

---

3. Session State Machine

A consultation session should follow a defined lifecycle.

unassigned
↓
assigned
↓
accepted_by_provider
↓
in_progress
↓
completed

Possible cancellation/failure paths may also be implemented where appropriate.

Required transitions

- Assign session
- Provider accepts session
- User starts session
- User ends session
- Provider ends session
- System ends session when balance runs out

Rules

- Invalid state transitions must not be allowed.
- Invalid transitions should return **HTTP 400** with a meaningful error.
- A user cannot have more than one active (`in_progress`) session at the same time.
- The per-minute cost should be fixed for the session once it starts.

---

4. Per-Minute Billing

An active session must be billed based on its per-minute cost.

For example:

Per-minute cost = ₹50
Session duration = 3 minutes

Total charge = ₹150

The billing logic should:

- Calculate the billable session time.
- Deduct the appropriate amount from the user's wallet.
- Track the amount/time already billed.
- Prevent the same minute from being charged more than once.

A billing operation should be available so that billing can be triggered periodically during an active session.

The implementation can use a simple synchronous billing mechanism for this assignment; production-grade background processing is **not required**.

---

5. Insufficient Balance

If the wallet does not have enough balance to pay for the next billable minute:

- Do not allow the wallet to become negative.
- End the session automatically.
- Mark the session as `completed`.
- Record that the session was ended by the system due to insufficient balance.
- Persist the actual talk/billable time.

Example:

Wallet balance = ₹40
Per-minute cost = ₹50

Next minute cannot be paid.

→ Session ends
→ Wallet remains ₹40
→ ended_by = system : Insufficient Balance

Authentication can use **JWT or simple token authentication**.

The system should distinguish between **USER** and **PROVIDER** roles.

---

6. Required Test Scenarios

The test suite must cover at least:

1. Insufficient balance

A user with less than 5 minutes' worth of balance cannot start a session.

2. Happy path

Recharge wallet
→ Create session
→ Provider accepts
→ Start session
→ Billing occurs for 2 minutes
→ End session

Verify that the wallet is debited correctly.

3. Balance runs out

During an active session, the wallet cannot cover the next minute.

Verify that:

- The session automatically ends.
- Wallet does not become negative.
- The correct ending reason is recorded.

4. Invalid state transition

Attempt an invalid action, such as ending a session before it has started.

Verify that the request returns **400**.

5. Duplicate recharge callback

Process the same successful recharge callback twice.

Verify that the wallet is credited only once.

6. Duplicate billing

Trigger billing more than once for the same billable minute.

Verify that the user is not charged twice.

---

8. Deliverables

1. Git repository containing the implementation.
2. README with setup and test instructions.
3. Automated test suite with all required tests passing.
4. Short state diagram or ER diagram.
5. Brief note explaining important implementation decisions.

Out of Scope

The following are not required:

- Real payment gateway integration
- Real-time chat
- WebRTC/audio/video
- Frontend/UI
- Admin panel
- Production-grade background workers
- Multi-currency support

What we are evaluating

We are primarily looking at how you handle:

State transitions → Wallet balance → Per-minute billing → Insufficient balance → Idempotency → Tests